

Module 6: Neural Networks and Optimization

A single merged treatment of Prince (UDL) Ch. 3–8,
with Goodfellow Ch. 7–8 as reference

Prepared for Tom Koreslesjak · companion to `NEC_thesis_syllabus.md`

June 2026

Abstract

This chapter builds the networks that everything downstream routes between: what a multilayer perceptron actually computes (a piecewise-linear function with astonishingly many pieces), why depth pays, how loss functions encode probabilistic commitments (and which commitments heavy-tailed targets break), backpropagation completed from the nine-tenths-derived state Foundations C left it in, the initialization derivations (Xavier, He) whose framework is already in place, and the practice layer — normalization, dropout, early stopping, clipping — each tied to a principle rather than presented as ritual. The destination: you can derive backprop, Xavier/He, and the batch-norm gradient from scratch, and you can build, train, and *debug* the MLP experts of the corrected NEC without copying a training loop on faith.

Contents

6.1 Shallow networks: what one hidden layer computes	3
6.2 Deep networks: why composition pays	3
6.3 Loss functions: probabilistic commitments, revisited for fat tails	4
6.4 Backpropagation, completed	5
6.5 Initialization: the derivations	6
6.6 Training practice: schedules, clipping, and reading loss curves	6
6.7 Normalization layers: batch norm, layer norm, and a financial warning	7
6.8 The regularization toolkit	8

6.9 Capstone: building and debugging the NEC experts	9
6.1 Exercises	10
Source map and what comes next	12

How this chapter was assembled (and what was cut)

Prince is the spine; Goodfellow is consulted, not read. This module is where the de-duplication ledger earns its keep, because the syllabus assigns here several items already done elsewhere in this book:

- *Done in Foundations C, cited not repeated:* SGD on the quadratic with its convergence rate and noise ball; momentum as a damped ODE; Adam with bias correction (Exercise C.3 discharged the syllabus exercise listed under this module); the matrix calculus and VJP cost-accounting behind backprop; the variance-propagation framework for initialization. Prince Ch. 6’s optimizer content is therefore compressed to cross-references plus what is new (schedules in practice, warmup, clipping).
- *Done in Module 4:* the softmax–cross-entropy gradient $\hat{p} - y$, its Hessian, and the through-one-linear-layer assembly (Exercise 4.5(c)) — the syllabus’s “CE + softmax through a linear layer” exercise here is upgraded rather than repeated (Exercise 6.10).
- *Done in Module 5:* model assessment. Prince Ch. 8 is consumed only for what is genuinely new to us — double descent — with everything else cross-referenced.
- *Prince Ch. 1–2:* cut to one sentence each; supervised learning and the loss-as-NLL principle are Foundations B §B.4 and §B.6.
- *Goodfellow Ch. 7 extras:* adversarial training and data augmentation get one honest paragraph (§6.8); neither is load-bearing for a tabular/sequence financial pipeline.
- *Universal-approximation theory:* statement and meaning only; the constructive proofs add no operative skill here.

Audit finding (assigned problems that do not exist). The syllabus assigns “Goodfellow Ch. 6, Exercises 6.1 and 6.2.” Goodfellow, Bengio & Courville contains *no end-of-chapter exercises* — verified against the local copy. Exercise 6.8 substitutes two problems covering the Ch. 6 material those phantom exercises presumably targeted (linear collapse of activation-free depth, and the XOR construction), flagged as substitutions.

Notation

Layers $\ell = 1, \dots, L$: pre-activations $f_\ell = W_\ell h_{\ell-1} + b_\ell$, activations $h_\ell = a[f_\ell]$ elementwise, $h_0 = x$, network output $f(x) = f_L$ (no activation on the output layer unless stated). $a'[\cdot]$ is the elementwise derivative; for $\text{ReLU}(z) = \max(z, 0)$, $a'[z] = \mathbf{1}[z > 0]$. Sensitivities $\delta_\ell = \nabla_{f_\ell} \mathcal{L}$. Widths n_ℓ ; $\odot$ is elementwise product. Prince’s ϕ, θ, ψ parameter notation is translated into this matrix notation throughout, including in the restated exercises.

6.1 Shallow networks: what one hidden layer computes

A shallow network with m hidden ReLU units computes

$$f(\mathbf{x}) = \phi_0 + \sum_{k=1}^m \phi_k \text{ReLU}(\boldsymbol{\theta}_k^\top \mathbf{x} + \theta_{k0}) : \quad (6.1.1)$$

a linear combination of *hinge functions*, each flat on one side of its hyperplane $\boldsymbol{\theta}_k^\top \mathbf{x} + \theta_{k0} = 0$ and linear on the other. The geometry to internalize: the m hyperplanes partition input space into convex cells; within each cell every ReLU is frozen (active or clipped), so f is *linear on each cell* — the network is a continuous piecewise-linear function, and “learning” is choosing where the creases go and what slopes to use between them. In one dimension, m hidden units buy up to m joints and $m + 1$ linear pieces (Prince’s figure 3.3; Exercise 6.7 works the bookkeeping); in p dimensions the cell count grows combinatorially — up to $\sum_{j=0}^p \binom{m}{j}$ regions, far more than m — which is the first hint that these models are more expressive than their parameter count suggests.

Two degenerate cases say what the nonlinearity is *for* (Exercise 6.7, after Prince 3.1): if a is linear, $a[z] = \psi_0 + \psi_1 z$, the whole of (6.1.1) collapses to a single affine function of $\mathbf{x}$ — composition of affine maps is affine — so the network, however wide, is linear regression in disguise; remove the activation entirely and the same collapse occurs layer-through-layer at any depth. *All* of a network’s expressive power beyond Module 4 lives in the kinks. The classical guarantee that the kinks suffice: the *universal approximation theorem* — a shallow network with enough hidden units can approximate any continuous function on a compact set to any accuracy. Read it as a license, not a strategy: it promises existence, says nothing about how many units, whether gradient descent finds them, or whether the fit generalizes — the bias–variance budget of Foundations B is not repealed, least of all at financial signal-to-noise.

6.2 Deep networks: why composition pays

Stack the construction: $\mathbf{h}_\ell = a[\mathbf{W}_\ell \mathbf{h}_{\ell-1} + \mathbf{b}_\ell]$. The output is still continuous piecewise-linear (compositions of piecewise-linear maps are piecewise-linear), so depth buys nothing in *kind*. What it buys is *economy*: each layer’s creases are drawn in the coordinates produced by the previous layer, so later layers fold earlier ones — a unit at depth two creates a crease that, pulled back through layer one’s folds, appears *multiple times* in input space. Repeated folding multiplies regions *multiplicatively* per layer rather than additively per unit: deep networks can carve exponentially (in depth) many linear regions out of the same parameter budget that buys a shallow network polynomially many. That is the honest, non-mystical version of “depth learns hierarchical features”: early layers build reusable coordinates, later layers spend them.

The practical counterweights, both already in your toolkit: deeper compositions are harder to *optimize* (Foundations C’s variance-propagation geometric applies forward to signals and backward to gradients — depth is where initialization, completed in §6.5, and normalization, §6.7, stop being pedantry), and at financial SNR the bias–variance budget caps useful capacity early. The NEC’s components are, by deliberate design, *shallow by deep-learning standards* — two-to-four-layer MLP experts, a modest recurrent encoder — and this section is the argument that this is

engineering judgment, not timidity: region-counting expressiveness is the resource you have in surplus; sample efficiency is the one you are starving for.

6.3 Loss functions: probabilistic commitments, revisited for fat tails

Foundations B §B.4 established the dictionary — training loss = negative log-likelihood of an observation model — and Module 4 exercised it for classification. Prince Ch. 5’s contribution is to make the dictionary *generative*: to design a loss, choose a distribution over outputs, let the network predict its parameters, take the NLL. Three entries matter for this pipeline.

Squared loss = fixed-variance Gaussian. $\mathcal{L}_i = (y_i - f(x_i))^2$ asserts $y \mid x \sim \mathcal{N}(f(x), \sigma^2)$ with σ constant. Both commitments are wrong for returns: tails are fat (Foundations B §B.7) and variance is regime-dependent — the entire premise of the thesis.

Heteroskedastic Gaussian. Let the network output two heads, mean $\mu(x)$ and variance $\sigma^2(x)$ (positivity via exp or softplus), with

$$\mathcal{L}_i = \frac{(y_i - \mu(x_i))^2}{2\sigma^2(x_i)} + \frac{1}{2} \log \sigma^2(x_i). \quad (6.3.1)$$

The first term is squared error *deflated by predicted risk*; the second prevents the cheap escape $\sigma \rightarrow \infty$. Within a mixture, this is exactly a Gaussian expert with learned per-sample variance — and (6.3.1) is what each NEC expert’s loss looks like once experts carry their own σ_k (the within-expert term of the mixture NLL (B.8.3)). A subtlety worth respecting: jointly training μ and σ can let the variance head “give up” on hard samples early (predict huge σ , kill the gradient on μ); warm-starting with frozen σ is the standard fix.

Robust losses. The Huber loss (quadratic near zero, linear in the tails) and the Student- t NLL (Foundations B, eq. B.7.1, as a loss) both cap the influence of extreme residuals — the gradient of squared loss grows linearly in the residual, so one –40% event dominates a batch gradient; Huber’s gradient saturates, and the t -NLL’s actually *decays* for extreme residuals (redescending influence). The decision for the thesis is not aesthetic: squared loss on raw returns delegates your gradient budget to tail events; the alternatives are rank-transformed targets (the OP pipeline’s choice — then squared loss is fine, the target is bounded), winsorized targets, or a robust likelihood. Choose one, state it, and keep it fixed across model comparisons — Module 5’s protocol applies to losses too.

Checkpoint exercise

Exercise 6.1 [Prince Probs. 5.1–5.2, extended]. [★★★ Core] (a) For the logistic sigmoid $\sigma(z) = 1/(1 + e^{-z})$, verify $\sigma(-\infty) = 0$, $\sigma(0) = \frac{1}{2}$, $\sigma(\infty) = 1$, and additionally prove $\sigma(-z) = 1 - \sigma(z)$ and that σ is strictly increasing. (b) For the binary cross-entropy of one pair, $\mathcal{L} = -(1 - y) \log(1 - \hat{p}) - y \log \hat{p}$ with $\hat{p} = \sigma(f(x))$: sketch (or characterize analytically — monotonicity, asymptotes, value at $\hat{p} = \frac{1}{2}$) the loss as a function of $\hat{p} \in [0, 1]$ for $y = 0$ and for $y = 1$. Where does the loss diverge, and why is that divergence harmless once the gradient is taken with respect to f rather than $\hat{p}$? (You proved the bounded-gradient fact in Module 4, eq. 4.8.3 — cite the mechanism.) (c) Heteroskedastic head check: for

the loss (6.3.1), derive $\partial \mathcal{L} / \partial \sigma^2$ and show the optimum at fixed μ is $\sigma^2 = (y - \mu)^2$ — the variance head wants to predict the squared residual. What failure mode does this enable early in training, and what is the standard fix (§6.3)?

6.4 Backpropagation, completed

Foundations C, Exercise C.4(c), derived the recursion’s skeleton on a two-layer bias-free network. Here is the full algorithm, with biases, in the notation every implementation uses. Forward: $\mathbf{f}_\ell = \mathbf{W}_\ell \mathbf{h}_{\ell-1} + \mathbf{b}_\ell$, $\mathbf{h}_\ell = a[\mathbf{f}_\ell]$, then $\mathcal{L}(\mathbf{f}_L, \mathbf{y})$. Backward, from $\delta_L = \nabla_{\mathbf{f}_L} \mathcal{L}$ (for squared loss, $2(\mathbf{f}_L - \mathbf{y})$; for fused softmax–CE, $\hat{\mathbf{p}} - \mathbf{y}$ by Module 4 §4.10):

$$\delta_{\ell-1} = a'[\mathbf{f}_{\ell-1}] \odot (\mathbf{W}_\ell^\top \delta_\ell), \quad \nabla_{\mathbf{W}_\ell} \mathcal{L} = \delta_\ell \mathbf{h}_{\ell-1}^\top, \quad \nabla_{\mathbf{b}_\ell} \mathcal{L} = \delta_\ell. \quad (6.4.1)$$

Three observations carry all the understanding. (i) Each line is a Foundations C identity: the weight gradient is the outer product of Example C.6.1 (sensitivity times input), the sensitivity update is a VJP through the linear map and the elementwise activation, and the bias gradient is the $x = 1$ special case. (ii) The cost analysis is Exercise C.7: one backward pass, same order of cost as the forward pass, in exchange for storing every $\mathbf{h}_\ell$ — which is why memory, not compute, is often the binding constraint, and why Module 7 truncates through time. (iii) For ReLU, a' is a 0/1 mask: backprop routes gradient only through units that were active forward — creases block gradient exactly where they block signal, the source of both sparsity and the “dead unit” failure mode (a unit whose pre-activation is negative for every training input receives zero gradient forever; He initialization and sane learning rates exist substantially to prevent this).

Exercise 6.2 (the syllabus’s centerpiece) re-derives (6.4.1) for an explicit two-layer network with squared loss and proves the general case by induction; Exercise 6.9 (Prince 7.1–7.2) forces the same content through brute-force scalar differentiation once — the pedagogical point being that you should do it by hand exactly once, watch the shared subexpressions multiply, and understand that backprop is nothing but their systematic reuse. The practical companion is *gradient checking*: compare analytic gradients against centered finite differences $(\mathcal{L}(\theta + \epsilon) - \mathcal{L}(\theta - \epsilon)) / 2\epsilon$ on a tiny network; the applied milestone (Exercise 6.11) does this against autograd.

Checkpoint exercise

Exercise 6.2 [Syllabus: backprop for the two-layer MLP, then induction]. [★★★★ Core] Let $\hat{\mathbf{y}} = \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$ with elementwise σ , and squared loss $\mathcal{L} = \|\mathbf{y} - \hat{\mathbf{y}}\|^2$. (a) Using the differential method (Foundations C §C.6), derive $\nabla_{\mathbf{W}_2} \mathcal{L}$, $\nabla_{\mathbf{b}_2} \mathcal{L}$, $\nabla_{\mathbf{W}_1} \mathcal{L}$, $\nabla_{\mathbf{b}_1} \mathcal{L}$, expressing each as products of forward-pass quantities ($\mathbf{x}$, $\mathbf{f}_1$, $\mathbf{h}_1$) and sensitivities. (b) Prove the general recursion (6.4.1) by induction on depth: assume it holds for the sub-network above layer ℓ , and show the inductive step is exactly one application of the fundamental identity $\nabla_{\mathbf{W}} g(\mathbf{W}\mathbf{h}) = \delta \mathbf{h}^\top$ plus one VJP. (c) List which quantities must be retained from the forward pass for the backward pass to run, and confirm the memory accounting of Foundations C, Exercise C.7(b).

6.5 Initialization: the derivations

Foundations C §C.7 established the framework: per-layer variance gain $n\sigma_W^2$ must equal one, forward and backward, or signals and gradients scale geometrically with depth. What remained was the activation bookkeeping, assigned by the syllabus to this module and now dischargeable in two steps.

Xavier/Glorot (Exercise 6.3): for activations roughly linear around zero (tanh near the origin), forward preservation requires $\sigma_W^2 = 1/n_{\text{in}}$ and backward preservation $\sigma_W^2 = 1/n_{\text{out}}$; irreconcilable unless the layer is square, so Glorot’s compromise takes the harmonic mean: $\sigma_W^2 = 2/(n_{\text{in}} + n_{\text{out}})$.

He (Exercise 6.4): ReLU is not linear around zero — it zeroes the negative half. For a symmetric zero-mean pre-activation $z \sim \mathcal{N}(0, s^2)$, the post-ReLU second moment is $\mathbb{E}[\text{ReLU}(z)^2] = \frac{1}{2} \mathbb{E}[z^2] = \frac{1}{2}s^2$ (the half-Gaussian moment — a two-line computation with Foundations B’s Gaussian integrals, and the exercise’s heart). Each ReLU layer therefore halves the variance that the next linear layer propagates, and the repair is to double the weight variance: $\sigma_W^2 = 2/n_{\text{in}}$. The factor of 2 in every `kaiming_normal_` call is this computation and nothing else.

Checkpoint exercise

Exercise 6.3 [Syllabus: Xavier/Glorot]. [★★★★ Core] Layer $\mathbf{h} = \mathbf{W}\mathbf{x}$, weights i.i.d. zero-mean variance σ_W^2 , inputs zero-mean variance σ_x^2 , independent; activation approximately identity near zero (the tanh regime). (a) Forward: from the Foundations B variance algebra, require $\text{Var}(h_i) = \sigma_x^2$ and derive $\sigma_W^2 = 1/n_{\text{in}}$. (b) Backward: apply the same argument to the sensitivity recursion $\delta_{\text{in}} = \mathbf{W}^\top \delta_{\text{out}}$ (mask $\approx \mathbf{1}$ in this regime) and derive $\sigma_W^2 = 1/n_{\text{out}}$. (c) Show the two conditions are incompatible unless $n_{\text{in}} = n_{\text{out}}$, and verify Glorot’s compromise $\sigma_W^2 = 2/(n_{\text{in}} + n_{\text{out}})$ is the harmonic mean of the two requirements. What residual geometric drift does the compromise accept in a deep, strongly rectangular network?

Exercise 6.4 [Syllabus: He initialization]. [★★★★ Core] (a) For $z \sim \mathcal{N}(0, s^2)$, compute $\mathbb{E}[\text{ReLU}(z)^2]$ by the half-Gaussian integral and show it equals $\frac{1}{2}s^2$. Compute also $\mathbb{E}[\text{ReLU}(z)] = s/\sqrt{2\pi}$ and note that ReLU outputs are not zero-mean — state why the variance-propagation argument survives this (what does the next layer’s zero-mean weight assumption do to the mean term?). (b) Conclude that with ReLU activations, forward variance preservation requires doubling Xavier’s forward choice: $\sigma_W^2 = 2/n_{\text{in}}$. (c) Sanity-check against Foundations C, Exercise C.5(a): with He init, what is the variance gain per layer of the 512-wide, 50-deep ReLU stack, and what would Xavier-forward give instead?

6.6 Training practice: schedules, clipping, and reading loss curves

The theory is Foundations C; this section is the residue of practice it did not cover. *Warmup*: the first few hundred steps of Adam run on EMA estimates built from almost no data (even bias-corrected, $\hat{v}$ is high-variance), and a freshly initialized network produces unusually large, unusually aligned gradients; ramping the learning rate from zero over a warmup window sidesteps both. *Schedules*: any decay respecting the two-regime logic of C §C.3 works; cosine-to-zero is the common convention; step decays are fine; what matters is that the floor (6.6.1)

shrinks when you need precision. *Gradient clipping*: replace $\mathbf{g}$ by $\mathbf{g} \cdot \min(1, c/\|\mathbf{g}\|)$ (norm clipping — direction-preserving; preferable to per-coordinate value clipping, which distorts direction). Two distinct justifications, both yours already: heavy-tailed *data* produce heavy-tailed per-batch gradients through the loss (Foundations B §B.7 — one extreme return, one extreme residual, one outsized gradient), and recurrent compositions produce occasional exploding gradients mechanically (the ρ^T amplification, formalized in Module 7). Clipping is a robustness device, not a hyperparameter to tune finely: set c near the typical gradient norm and forget it.

$$(\text{noise-floor reference, from C}): \quad \mathbb{E} \|e_\infty\|^2 \propto \eta \sigma_g^2 / \lambda_{\min}. \quad (6.6.1)$$

Reading loss curves is diagnosis with this chapter’s vocabulary: divergence after a learning-rate change (stability ceiling, C §C.2); a plateau that a decay step breaks (noise floor); training loss falling while purged-validation rank-IC degrades (variance term — stop earlier or regularize harder); training loss stuck at initialization levels (dead ReLUs, broken init, or a wiring bug — the overfit-one-batch test of §6.9 discriminates).

6.7 Normalization layers: batch norm, layer norm, and a financial warning

Batch normalization standardizes each unit’s pre-activation across the minibatch, then restores expressiveness with learned scale and shift:

$$\mu = \frac{1}{B} \sum_{i=1}^B x_i, \quad \sigma^2 = \frac{1}{B} \sum_{i=1}^B (x_i - \mu)^2, \quad \hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta, \quad (6.7.1)$$

per unit, with (γ, β) trained. Why it helps is less settled than how: the original “internal covariate shift” story has not survived scrutiny; the better-supported account is that BN smooths and re-conditions the optimization landscape (each layer sees inputs of controlled scale regardless of what upstream layers did — runtime enforcement of the gain-one condition that initialization only arranges at $t = 0$), permitting larger learning rates. Two operational facts cause most BN bugs. *Train/eval asymmetry*: at evaluation there is no batch, so BN uses running averages of (μ, σ^2) accumulated during training — forgetting `model.eval()` is the classic silent error, and a distribution shift between training batches and deployment data makes the running statistics simply wrong. *Cross-sample coupling*: (6.7.1) makes every sample’s output depend on every other sample in its batch — the property the gradient inherits, which is what makes Exercise 6.5 (the syllabus’s batch-norm gradient) genuinely instructive: $\partial \mathcal{L} / \partial x_i$ picks up three terms, the direct path plus the paths through μ and σ^2 , and the result couples all B gradients.

NEC connection

The cross-sample coupling is not a numerical curiosity in finance — it is a leakage channel. If a minibatch mixes dates, BN’s batch statistics let each sample’s forward pass see *other dates’* feature levels, including future ones; with date-blocked batches the statistics become date-summaries, which is better but still couples assets within a date (sometimes desirable — it resembles cross-sectional standardization — but then it must be *consistent* between training and the strictly per-date evaluation pipeline). The clean resolution for this pipeline: *layer*

normalization — normalize across each sample’s own features, $\mu_i = \frac{1}{n} \sum_j x_{ij}$, no cross-sample terms, no running statistics, no train/eval asymmetry — which is also the standard choice inside recurrent and attention models (Module 7’s encoder) for exactly the no-batch-coupling reason. Default to LayerNorm everywhere in the NEC; admit BatchNorm only with date-blocked batches and a written argument that its statistics match the evaluation-time transform.

Checkpoint exercise

Exercise 6.5 [Syllabus: the batch-norm gradient]. [★★★★ Core] For one unit with batch values $x_1, \dots, x_B$, statistics and outputs as in (6.7.1) (treat $\epsilon = 0$, fold γ into the upstream gradient so $g_i = \partial \mathcal{L} / \partial \hat{x}_i$): (a) Identify the three paths by which x_i influences $\mathcal{L}$ (directly through $\hat{x}_i$; through μ ; through σ^2), and write the total derivative accordingly. (b) Derive the closed form

$$\frac{\partial \mathcal{L}}{\partial x_i} = \frac{1}{\sigma} \left(g_i - \frac{1}{B} \sum_{j=1}^B g_j - \hat{x}_i \frac{1}{B} \sum_{j=1}^B g_j \hat{x}_j \right).$$

(c) Verify two structural checks: the batch of gradients sums to zero, and it is orthogonal to $(\hat{x}_1, \dots, \hat{x}_B)$ — BN’s gradient is the upstream gradient with its batch-mean and its $\hat{x}$ -component projected out. Explain why both must hold (what invariances does (6.7.1) have?). (d) One sentence: connect the cross-sample terms to the leakage warning of §6.7.

6.8 The regularization toolkit

Every device below is a bias–variance trade (Foundations B, eq. B.5.1) with its own mechanism; the craft is knowing which mechanism, so the devices are combined rather than stacked superstitiously.

Weight decay. Fully understood already: MAP with a Gaussian prior (Module 4 §4.5), implemented as AdamW’s decoupled decay (Foundations C §C.5), doubling as conditioning repair and as the gate’s separation cure. Nothing to add but the reminder that it is the *one* regularizer with a clean probabilistic semantics, hence the first knob to reach for.

Early stopping. Halt training when validation performance (for us: purged-validation rank-IC) stops improving. This is not an absence of regularization but a form of it: on the quadratic model problem, t steps of gradient descent from zero shrink each eigenmode by $1 - (1 - \eta \lambda_j)^t$ — a spectral filter that passes high-curvature directions and suppresses low-curvature ones, *the same shape* as ridge’s $\hat{\lambda}_j$ -shrinkage with the correspondence $\lambda_{\text{ridge}} \approx 1/(\eta t)$. Exercise 6.6 derives this correspondence exactly; its content is why “train less” and “penalize more” are interchangeable first-order responses to overfitting, and why early stopping patience belongs in the hyperparameter registry of Module 5’s protocol (it *is* a tuned complexity parameter).

Dropout. During training, zero each hidden unit independently with probability q and scale survivors by $1/(1 - q)$ (inverted dropout, so evaluation runs the unmodified network). Three readings, in decreasing solidity: it is multiplicative noise injection that prevents co-adaptation

(units cannot rely on specific partners that may vanish); it approximates training an exponential ensemble of subnetworks with shared weights, evaluated at test time by weight scaling; and for linear regression, dropout on inputs is *exactly* equivalent to a data-dependent ridge penalty — the cleanest available statement that noise is regularization. On small tabular financial models, modest dropout ($q \approx 0.1\text{--}0.3$) on hidden layers is a workhorse; on the gate, prefer weight decay (dropout noise in routing destabilizes the mixture’s responsibilities).

Augmentation and adversarial training (Goodfellow Ch. 7, one honest paragraph). Augmentation manufactures invariances by transforming inputs label-free — transformative in vision, weak in finance because the invariances of returns are unknown (what is the “rotated” version of a momentum signal?); the nearest financial analogues are feature noising and resampled (block-bootstrap, Foundations B §B.4.5) training windows, both defensible, neither standard. Adversarial training optimizes against worst-case perturbations; its pipeline relevance is conceptual — a reminder that models fit at one data distribution fail off it, which finance operationalizes as regime shift rather than pixel attacks.

Double descent (Prince Ch. 8’s genuinely new item). Heavily overparameterized networks can interpolate the training data and *still* generalize, with test error descending a second time past the classical U-curve’s interpolation peak — a real phenomenon, theoretically linked to implicit regularization of gradient methods in overparameterized regimes. Treat it as a flag planted for intellectual honesty, not an invitation: the regime requires clean labels and abundant signal, and a return panel at $IC \approx 0.03$ is the opposite of both. The NEC lives on the classical side of the curve, where Foundations B’s budget arithmetic governs and Module 5’s validation discipline decides.

Checkpoint exercise

Exercise 6.6 [Supplementary: early stopping is spectral ridge]. [★★★★ Core] *Gradient descent from $\beta_0 = \mathbf{0}$ on the quadratic model problem (Foundations C §C.2), step $\eta < 1/\lambda_{\max}$. (a) Show in the eigenbasis that after t steps, $\tilde{\beta}_{j,t} = (1 - (1 - \eta\lambda_j)^t) \tilde{\beta}_j^*$: a shrinkage factor per mode, increasing in λ_j and in t . (b) Place it beside ridge’s factor $\lambda_j / (\lambda_j + \lambda_{\text{ridge}})$ (Foundations A / Module 4): show both filters pass high-curvature directions and suppress low-curvature ones, and by matching either the small- λ_j slopes or the half-shrinkage points, derive the correspondence $\lambda_{\text{ridge}} \approx 1/(\eta t)$. (c) Conclusions, one sentence each: why “longer training” moves along the same complexity axis as “weaker penalty”; and why early-stopping patience must therefore be logged in Module 5’s trial registry as a hyperparameter.*

6.9 Capstone: building and debugging the NEC experts

The corrected NEC’s experts are MLPs of the form now fully specified: two-to-four layers, ReLU, He init, LayerNorm, modest dropout, AdamW with warmup-then-decay, norm clipping, loss from §6.3 (squared on rank-transformed targets, or heteroskedastic Gaussian once per-expert variances enter), trained under the mixture objective whose gradients are (B.8.5)–(B.8.6) and judged by Module 5’s protocol. What this section adds is the *debugging liturgy* — ordered, cheap, and diagnostic:

1. *Shape audit.* Print every tensor shape through one forward pass; the matrix-calculus shape discipline of Foundations C §C.6 catches transposition bugs before any training run.
2. *Gradient check.* Centered finite differences vs. analytic gradients on a tiny copy of the network (Exercise 6.11); run once after any hand-written loss or layer.
3. *Overfit one batch.* A correct model-loss-optimizer triple must drive one minibatch's loss to ≈ 0 ; failure localizes the bug to wiring, not data or capacity. (Passing says nothing about generalization — this is a plumbing test.)
4. *Initialization sanity.* At $t = 0$: loss should equal its theoretical chance value (e.g. $\log K$ for a K -class CE — a uniform gate; Foundations B §B.6), activations should have unit-scale variance layer through layer (§6.5); a factor-of-ten anomaly at $t = 0$ is an init or normalization bug.
5. *Loss-curve reading.* The grammar of §6.6; for the mixture, add the gate dashboards — entropy, utilization, responsibilities — from Modules 4 and B, because expert-side bugs masquerade as gate collapse and vice versa.
6. *Ablate toward the trivial.* When the full mixture misbehaves, fall back: single expert (no gate), then linear expert, then ridge — the Module 4 baseline. The first rung of the ladder that works brackets the fault.

Checkpoint (from the syllabus, with the ledger applied). Closed book: derive backprop (6.4.1) for two layers and by induction for L (Exercise 6.2); derive Xavier and He, including the half-Gaussian factor (Exercises 6.3–6.4); derive the batch-norm gradient with all three coupling terms (Exercise 6.5); and — discharged in Foundations C, recite rather than re-derive — SGD's convergence and noise ball, momentum's ODE, Adam's bias correction. Practically: you can build, train, and debug a small PyTorch model via the liturgy above without copying a training loop blindly. If all of that is comfortable, Module 6 has done its job, and the sequence encoder of Module 7 is the only architectural piece still missing.

6.10 Exercises

Importance labels. Each exercise carries a priority badge for the NEC/MoE thesis track, reflecting how load-bearing it is — *not* how hard it is. [★★★ Core] are the chapter's must-derive spine: the loss heads your experts will actually train under (6.1 — its part (c) is the per-expert variance head and its failure mode), backpropagation (6.2), the two initializations your ReLU experts and encoder depend on, Xavier (6.3) and He (6.4), the batch-norm gradient (6.5) — the hardest exercise here, kept Core because its cross-sample coupling is the leakage trap of §6.7 and is the reason the NEC defaults to LayerNorm — and early-stopping-as-spectral-ridge (6.6), Core because patience is a complexity dial the Module 5 trial registry must log. [★★ Worth it] are high value a notch below, including the do-it-once brute-force backprop (6.9) and the applied PyTorch milestone (6.11), worth doing when you start implementing. [★ Low priority] is foundational-but-familiar (6.8: affine collapse and XOR). The optimization derivations the syllabus also lists here — SGD, momentum, Adam — were done once in Foundations C and are recited, not repeated.

Questions only, as established. All assigned Prince problems appear, restated in this book’s notation and made self-contained (figure references replaced by explicit constructions). The assigned “Goodfellow Ch. 6 Exercises 6.1–6.2” do not exist in that book (see preface); Exercise 6.8 substitutes. The syllabus’s SGD-quadratic, momentum-ODE, and Adam exercises for this module were discharged in Foundations C (Exercises C.1–C.3 and C.6) and are not repeated. **Core exercises ([★★★ Core]) sit inline in the chapter body as checkpoint exercises, at the end of the section they test; this section collects the supplementary exercises only.**

On shallow and deep networks (Sections 6.1–6.2)

Exercise 6.7 [Prince Probs. 3.1–3.2, extended with 3.3’s slopes]. [★★ Worth it] *Take the one-dimensional, three-hidden-unit ReLU network $y = \phi_0 + \sum_{k=1}^3 \phi_k \text{ReLU}(\theta_{k0} + \theta_{k1}x)$, with parameters chosen so the three joint locations $x_k = -\theta_{k0}/\theta_{k1}$ are distinct, creating four linear regions. (a) What mapping results if the activation is linear, $a[z] = \psi_0 + \psi_1 z$? What if it is removed, $a[z] = z$? Prove the collapse in both cases. (b) For each of the four regions, identify which hidden units are active (pass their input) and which are clipped (output zero), assuming each $\theta_{k1} > 0$. (c) Derive the joint positions and the slope of y in each region in terms of the parameters, and verify the slopes change by $\phi_k \theta_{k1}$ at joint k — each unit contributes its “hinge” exactly once.*

Exercise 6.8 [Substitution for the nonexistent Goodfellow Ch. 6 exercises]. [★ Low priority] (a) *Prove the depth-collapse theorem behind Exercise 6.7(a) in general: a network of L layers with identity (or any affine) activation computes a single affine map, for any widths. Conclude in one sentence why “deep linear networks” nonetheless remain interesting to optimization theorists though not to function approximators. (b) The XOR function on $\{0,1\}^2$ (output 1 iff inputs differ): prove no affine classifier $\text{sign}(w_1 x_1 + w_2 x_2 + b)$ computes it, then construct an explicit two-hidden-unit ReLU network that does, verifying all four input cases. (This is the canonical Goodfellow §6.1 worked example, recast as a problem.)*

On backpropagation (Section 6.4)

Exercise 6.9 [Prince Probs. 7.1–7.2: backprop by brute force, once]. [★★ Worth it] *Take the scalar two-layer, two-units-per-layer network*

$$y = \phi_0 + \phi_1 a[\psi_{01} + \psi_{11}a[\theta_{01} + \theta_{11}x] + \psi_{21}a[\theta_{02} + \theta_{12}x]] + \phi_2 a[\psi_{02} + \psi_{12}a[\theta_{01} + \theta_{11}x] + \psi_{22}a[\theta_{02} + \theta_{12}x]],$$

with $a = \text{ReLU}$, so $a'[z] = \mathbf{1}[z > 0]$. (a) Compute $\partial y / \partial \phi_\bullet$, $\partial y / \partial \psi_\bullet$, $\partial y / \partial \theta_\bullet$ for all 13 parameters directly, without the backprop algorithm. Mark every subexpression you compute more than once. (b) For a depth-3 chain $f_0 \rightarrow h_1 \rightarrow f_1 \rightarrow h_2 \rightarrow f_2 \rightarrow h_3 \rightarrow f_3 \rightarrow \ell$ written in our notation, give explicit expressions for each factor type appearing in the chain-rule products: $\partial \ell / \partial \mathbf{f}_3$, $\partial \mathbf{h}_\ell / \partial \mathbf{f}_{\ell-1}$ (a weight matrix — which one, transposed or not?), and $\partial \mathbf{f}_\ell / \partial \mathbf{h}_\ell$ (a diagonal mask — of what?). Reconcile with the boxed recursion (6.4.1). (c) One paragraph: state precisely what backpropagation reuses that your part-(a) computation recomputed, and how the cost ratio scales with depth and width (Foundations C, Exercise C.7).

Exercise 6.10 [Syllabus, upgraded: cross-entropy through the last two layers]. [★★ Worth it] *The fused softmax–CE block hands backprop the starting sensitivity $\delta_L = \hat{\mathbf{p}} - \mathbf{y}$ (Module 4 §4.10; do not*

re-derive). (a) Push it through the final linear layer: recover $\nabla_{\mathbf{w}_L} = (\hat{\mathbf{p}} - \mathbf{y}) \mathbf{h}_{L-1}^\top$ — and note this is Module 4’s Exercise 4.5(c), now seen as the first step of (6.4.1). (b) Continue one layer deeper through a ReLU hidden layer: derive $\nabla_{\mathbf{w}_{L-1}} \mathcal{L}$ explicitly, and identify where the 0/1 mask enters. (c) Explain in two sentences why implementations expose “cross-entropy-with-logits” as a single op: which two numerically dangerous intermediates does the fusion never materialize (Module 4 §4.10; Foundations B §B.6.3)?

Applied milestone (Section 6.9)

Exercise 6.11 [Applied milestone — optional, computational]. [★★ **Worth it**] The syllabus’s practical milestone; the one computational exercise of the chapter, consistent with house policy. Implement the two-layer MLP of Exercise 6.2 in PyTorch. (i) Verify your hand-derived gradients against `torch.autograd` on a small random input (centered finite differences as a third check). (ii) Run the debugging liturgy of §6.9 on it: shape audit, overfit-one-batch, $t = 0$ loss against the chance value. (iii) Break it deliberately — a forgotten `model.eval()` with a BatchNorm layer inserted — and observe which diagnostic catches it.

Source map and what comes next

Section	Primary source(s)	Feeds directly into
6.1 Shallow nets, regions	Prince Ch. 3	expert architecture intuition
6.2 Depth and folding	Prince Ch. 4	why NEC components stay shallow
6.3 Losses as likelihoods	Prince Ch. 5 + Fnd. B §B.7	expert losses; mixture NLL (Module 8)
6.4 Backprop completed	Prince Ch. 7 + Fnd. C	BPTT (Module 7)
6.5 Xavier and He	Prince Ch. 7 + Fnd. C §C.7	every <code>init</code> call in the code
6.6 Schedules, clipping	Prince Ch. 6 + GF §8.1–8.3	the NEC training loop
6.7 BatchNorm vs LayerNorm	Prince Ch. 7 / GF §8 + own	encoder normalization (Module 7)
6.8 Regularization toolkit	Prince Ch. 8, GF Ch. 7	hyperparameter registry (Module 5)
6.9 Debugging liturgy	synthesis	every experiment from here on

One architectural piece remains before the thesis’s core: the sequence encoder. Module 7 derives backpropagation *through time* as (6.4.1) unrolled along a sequence, proves the vanishing/exploding-gradient theorem as the third appearance of this book’s geometric-amplification motif (after GD modes and depth-variance), and dissects the GRU and LSTM gates as engineered solutions — ending with a line-by-line mathematical reading of the LSTM encoder `C_L` and classifier head `C_F` in the existing NEC code, the components the corrected model will inherit or replace.